



ENGIN 604 INTRODUCTION TO PYTHON FOR FINANCE

ITERATIVE CONTROL STRUCTURES - SOLUTION

Professor: *Gabriel E. Cabrera*
Teaching Assistant: *Alex Den Braber*



1 *Loops with & without range()*

1. Build a list containing the names of the following technology companies: 'Facebook', 'Apple', 'Amazon' and 'Netflix'.

- a. Print to the console each element in the list.

```
list1 = ['Facebook', 'Apple', 'Amazon', 'Netflix']

for tech in list1:
    print('Company: '+tech)
```

```
## Company:Facebook
## Company:Apple
## Company:Amazon
## Company:Netflix
```

- b. Print to the console each element in the list with its corresponding index.

```
for i, tech in enumerate(list1):
    print('index: '+str(i)+' '+'Company: '+tech)
```

```
## index:0,Company:Facebook
## index:1,Company:Apple
## index:2,Company:Amazon
## index:3,Company:Netflix
```

- c. Print to the console each letter that makes up each element (company) in the list.

```
for tech in list1:
    for i, letter in enumerate(tech):
        print('n°: '+str(i)+' '+'Letter: '+letter+' '+'Company: '+tech)
```

```
## n°:0,Letter:F,Company:Facebook
## n°:1,Letter:a,Company:Facebook
## n°:2,Letter:c,Company:Facebook
## n°:3,Letter:e,Company:Facebook
## n°:4,Letter:b,Company:Facebook
## n°:5,Letter:o,Company:Facebook
## n°:6,Letter:o,Company:Facebook
## n°:7,Letter:k,Company:Facebook
## n°:0,Letter:A,Company:Apple
```

```
## n°:1,Letter:p,Company:Apple
## n°:2,Letter:p,Company:Apple
## n°:3,Letter:l,Company:Apple
## n°:4,Letter:e,Company:Apple
## n°:0,Letter:A,Company:Amazon
## n°:1,Letter:m,Company:Amazon
## n°:2,Letter:a,Company:Amazon
## n°:3,Letter:z,Company:Amazon
## n°:4,Letter:o,Company:Amazon
## n°:5,Letter:n,Company:Amazon
## n°:0,Letter:N,Company:Netflix
## n°:1,Letter:e,Company:Netflix
## n°:2,Letter:t,Company:Netflix
## n°:3,Letter:f,Company:Netflix
## n°:4,Letter:l,Company:Netflix
## n°:5,Letter:i,Company:Netflix
## n°:6,Letter:x,Company:Netflix
```

2. Generate a sequence from 0 to 10 and print each value to the console.

```
for num in range(0,11):
    print(num)
```

```
## 0
## 1
## 2
## 3
## 4
## 5
## 6
## 7
## 8
## 9
## 10
```

3. Generate a sequence from 0 to 10 and print each value to the console in descending order.

```
for num in reversed(range(0,11)):
    print(num)
```

```
## 10
## 9
## 8
## 7
## 6
## 5
## 4
## 3
## 2
## 1
## 0
```

2 Conditionals

1. Build the following sequence of numbers including the `None` values: 1, 2, `None`, 4, 5, 6, `None`. Then, using a *loop*, add up the elements that are numeric (do not consider `None`)

```
seq = [1, 2, None, 4, 5, 6, None]

total = 0 # variable equal to zero

for j in seq:
    if j == None:
        continue
    else:
        total += j

print('The total sum is: '+str(total))
```

```
## The total sum is: 18
```

2. Build the following sequence of numbers including the `None` values: 1, 2, `None`, 4, 5, 6, `None`. Then, using a *loop*, add up all the numeric elements before the first `none`.

```
seq1 = [1, 2, None, 4, 5, 6, None]

# variable equal to zero
total1 = 0

for j in seq1:
    if j == None:
        break
    else:
        total1 += j

print('The total sum is: '+str(total1))
```

```
## The total sum is: 3
```

3. Build the following sequence of numbers: 1, 2, 3, 4, 5, 6, 7. Then, using a *loop*, add up the elements smaller than the number 5.

```
seq2 = list(range(1,8))

# way 1
total2 = 0

for j in seq2:
    if j < 5:
        total2 += j
    elif j == 5:
        print(str(j)+' is equal to 5')
    else:
        print(str(j)+' is greater than 5')
```

```
## 5 is equal to 5
## 6 is greater than 5
## 7 is greater than 5
```

```
total2
```

```
## 10
```

4. Add up all the numbers from 0 to 15 that are multiples of 3 or 5.

```
seq2 = range(16)

total3 = 0

for k in seq2:
    if (k%3 == 0) or (k%5 == 0):
        total3 += k
    else:
        print(str(k)+' is not a multiple of 3 or 5')
```

```
## 1 is not a multiple of 3 or 5
## 2 is not a multiple of 3 or 5
## 4 is not a multiple of 3 or 5
## 7 is not a multiple of 3 or 5
## 8 is not a multiple of 3 or 5
## 11 is not a multiple of 3 or 5
## 13 is not a multiple of 3 or 5
## 14 is not a multiple of 3 or 5
```

```
total3
```

```
## 60
```

5. Build a *loop* that generates the Fibonacci sequence, which is characterized by each number being the sum of the two previous numbers. For example, the first 14 numbers of the sequence are: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233.

```
fib = []

fib.append(0)
fib.append(1)

x = 1

while x <= 12:
    res = fib[x] + fib[x - 1]
    fib.append(res)
    x += 1
```

```
fib
```

```
## [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233]
```

3 *Comprehension*

A “*list comprehension*” allows you to build new lists (`list`) by filtering the elements of a collection (sequence) with a single expression. Its traditional form is the following:

```
[expr for val in collection if condition]
```

the above is equivalent to:

```
res = []
for val in collection:
    if condition:
        result.append(expr)
```

For example:

```
list2 = ['Facebook', 'Apple', 'Amazon', 'Netflix']
```

```
list2_title = [elem.upper() for elem in list2]  
list2_title
```

```
## ['FACEBOOK', 'APPLE', 'AMAZON', 'NETFLIX']
```

Which in its standard form is equivalent to:

```
list2_title2 = []
```

```
for elem in list2:  
    list2_title2.append(elem.upper())
```

```
list2_title2
```

```
## ['FACEBOOK', 'APPLE', 'AMAZON', 'NETFLIX']
```

You can even add the `else` conditional, which changes the structure of the *list comprehension* slightly, but the goal is the same:

```
[num if (num%3 == 0) or (num%5 == 0) else None for num in range(16)]
```

```
## [0, None, None, 3, None, 5, 6, None, None, 9, 10, None, 12, None, None, 15]
```

Which in its standard form is equivalent to:

```
res2 = []
```

```
for num in range(16):  
    if (num%3 == 0) or (num%5 == 0):  
        res2.append(num)  
    else:  
        res2.append(None)
```

```
res2
```

```
## [0, None, None, 3, None, 5, 6, None, None, 9, 10, None, 12, None, None, 15]
```

It can also be used to “flatten” nested lists:

```
nested_list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
[sub_elem for elem in nested_list for sub_elem in elem]
```

```
## [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Which in its standard form is equivalent to:

```
flattened_list = []
```

```
for elem in nested_list:  
    for sub_elem in elem:  
        flattened_list.append(sub_elem)
```

```
flattened_list
```

```
## [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

To see *comprehension* for dictionaries (`dict`) and tuples (`tuple`), see pages 67 and 68 of the book *Python for Data Analysis: Data Wrangling with Pandas, Numpy, and Ipython* by Wes McKinney.